

Building Production-Ready Agents

Breakout Lab: Deploy and Inspect a Governed Agent

Amazon Bedrock AgentCore

Lab Overview (1/2)

Goal: Deploy a pre-configured agent on AgentCore Runtime and verify its identity, tool access, memory, and observability features.

- **Duration:** 45 minutes
- **Format:** Pair work (One person per team)
- **Prerequisites:** AWS sandbox access, Python 3.10+, AWS CLI

Lab Overview (2/2)

NOTE

All supporting resources (Gateway, Memory, IAM roles, policies) are pre-created by the instructor. You will implement, deploy, and operate the Runtime agent.

IAM: Identity and Access Management — the AWS service that controls who, or what, can access which resources.

Lab Tasks

1. Load team credentials and identify architecture components
2. Inspect the restricted IAM policy on the execution role
3. Implement the agent logic (three TODOs in `agent.py`)
4. Deploy the agent to Runtime
5. Invoke the read-only tool through Gateway
6. Verify short-term memory continuity across sessions
7. Inspect the end-to-end trace in CloudWatch

Setup: Load Credentials

Load Your Team Credentials (1/2)

The instructor sent a presigned S3 link to `team-XX.env`. Save it outside this repo (e.g. your home directory), then load and verify:

```
source ~/team-XX.env  
aws sts get-caller-identity
```

Open `team-XX.env` in VS Code to examine its contents. It already sets:

```
AWS_ACCESS_KEY_ID , AWS_SECRET_ACCESS_KEY , AWS_DEFAULT_REGION ,  
EXECUTION_ROLE_ARN , S3_BUCKET , AGENT_NAME , S3_KEY ,  
GATEWAY_URL , GATEWAY_TOOL_NAME , MEMORY_ID
```

Load Your Team Credentials (2/2)

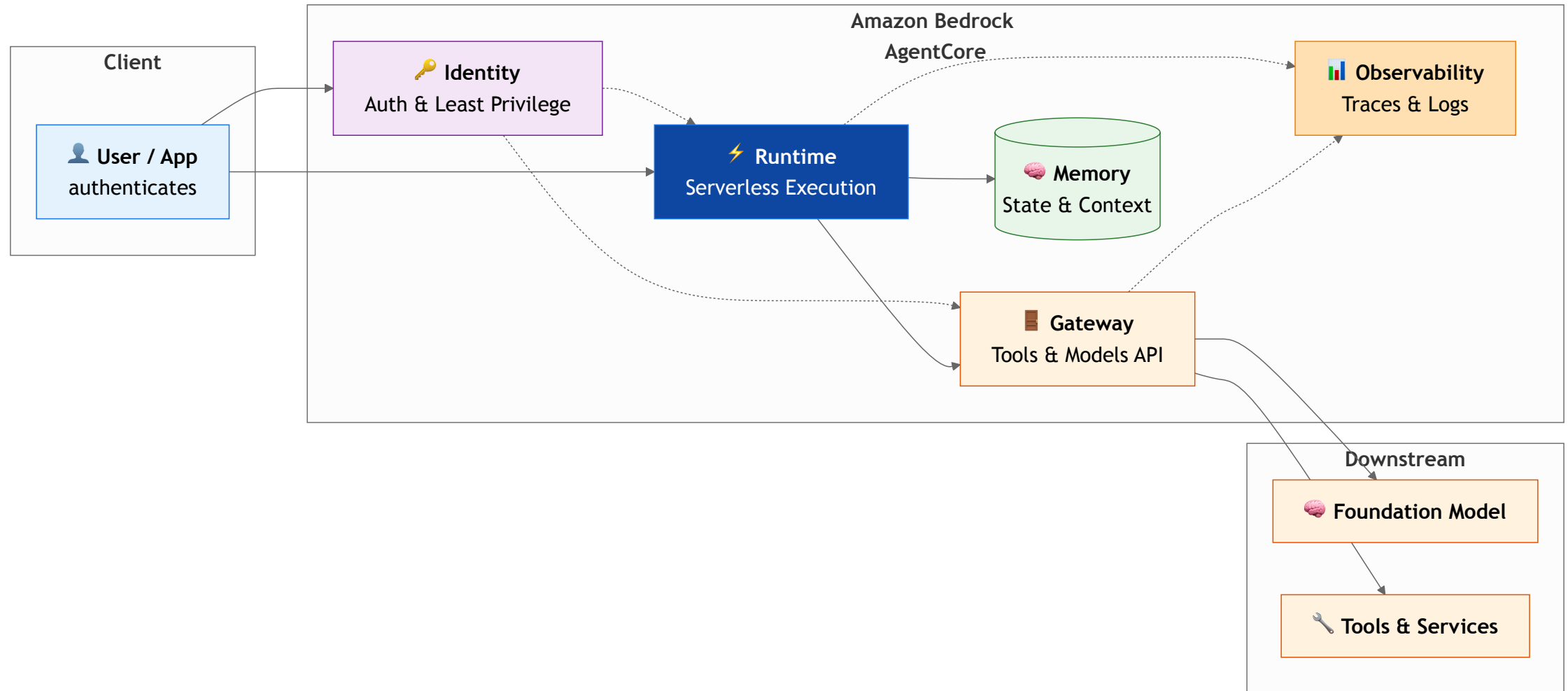
Export only the values **not** included in `team-XX.env` :

```
export AWS_REGION=us-east-1
export ACTOR_ID=<instructor-provided>
export MEMORY_SESSION_ID=<instructor-provided>
export ORDER_ID=ORD-1001
export EXPECTED_ORDER_STATUS=SHIPPED
```

CAUTION

Do not re-export `EXECUTION_ROLE_ARN` , `S3_BUCKET` , `AGENT_NAME` , or other values already in `team-XX.env` . Re-exporting overwrites the real values and causes `AccessDenied` or deployment failures.

Identify the Components (1/2)



Identify the Components (2/2)

On the diagram, locate:

- Caller (human user)
- Runtime endpoint (your agent)
- Execution role (ProdAgentRole)
- Gateway target (read-only tool)
- Memory resource
- CloudWatch (telemetry destination)

Checkpoint 1: Identity and Policy

Inspect the IAM Policy (1/2)

```
aws iam list-role-policies --role-name "${EXECUTION_ROLE_ARN##*/}"  
aws iam get-role-policy --role-name "${EXECUTION_ROLE_ARN##*/}" \  
    --policy-name AgentCoreExecutionPolicy
```

Inspect the IAM Policy (2/2)

Restricted production policy scopes the role to:

- `bedrock:InvokeModel` on the assigned model only
- `bedrock-agentcore:InvokeGateway` on the assigned Gateway
- Memory event read/write on the assigned Memory
- Telemetry actions on the assigned log and trace destinations

NOTE

The restricted policy is already attached as an inline role policy by provisioning. Students are not granted `iam:PutRolePolicy`. If it is missing, ask the instructor to re-provision.

Checkpoint 2: Implement the Agent Logic

Complete the Three TODOs (1/2)

Open `start/agent.py` and implement:

1. `call_gateway_tool` — Invoke the Gateway MCP tool using SigV4Auth and `urllib.request`
2. `load_memory_turns` — Retrieve conversation history from AgentCore Memory using `list_events`
3. `write_memory_turn` — Write new messages to AgentCore Memory using `create_event`

Complete the Three TODOs (2/2)

Developer Prompt (for coding assistants):

Complete the three TODOs in `start/agent.py`. Use `boto3 SigV4Auth` and `urllib.request` for the Gateway tool call. Use the `bedrock-agentcore` `boto3` client for `list_events` and `create_event`. Ensure the returned memory turns are formatted correctly for the Bedrock Converse API.

TIP

Review `check_gateway.py` for a working SigV4 signing example you can adapt for TODO 1.

Checkpoint 3: Deploy the Agent

Deploy to Runtime (1/2)

```
cd day1/labs/05-operating-the-environment/start/  
python3 deploy_agent.py
```

The script will:

1. Validate the prebuilt ZIP in your assigned S3 (Simple Storage Service) bucket
2. Create or update the assigned Runtime with the restricted role
3. Wait for the returned Runtime version to reach **READY**

Deploy to Runtime (2/2)

After deployment, load the generated environment file:

```
source deploy.env
```

This sets `AGENT_RUNTIME_ID`, `AGENT_RUNTIME_VERSION`, and `AGENT_RUNTIME_ARN` for the remaining checkpoints. Re-running `deploy_agent.py` auto-detects and updates the existing Runtime.

TIP

If deployment fails, check: IAM role attached, S3 bucket access, region matches your sandbox.

Checkpoint 4: Invoke Through Gateway

Invoke the Read-Only Tool

```
python3 check_gateway.py
```

What happens:

- Your student process signs a direct Gateway checkpoint with SigV4 (AWS Signature Version 4)
- Gateway validates `bedrock-agentcore:InvokeGateway`
- MCP (Model Context Protocol) `tools/call` invokes the pre-created read-only target
- The script rejects JSON-RPC errors and checks order plus status

VERIFY

Confirm `GATEWAY_OK` and the synthetic order `ORD-1001`. A failure must exit nonzero.

Checkpoint 5: Verify Memory Continuity

Test Short-Term Memory

Run `python3 verify_memory.py`. It uses different Runtime session IDs with the same Memory `actorId` and `sessionId`:

Invocation 1: "Check order ORD-1001 and remember Salesforce case 00001042."

Invocation 2: "Which Salesforce case did I ask you to remember?"

Expected behavior:

- Invocation 1: Runtime calls the retail tool and writes synthetic case context to Memory
- Invocation 2: Agent retrieves case `00001042`; `ListEvents` confirms stored events

KEY INSIGHT

The same actorId + sessionId retrieves the same event stream. IAM and application authorization must prevent callers from spoofing another actorId.

Checkpoint 6: Inspect the Trace

CloudWatch Trace Inspection

Navigate to **CloudWatch** → **Generative AI Observability** → **AgentCore**

Find your trace and identify:

- Total request duration (`agent.run` span)
- Token usage (input + output per LLM call)
- Tool invocations (which tools, how long)
- Memory operations (reads/writes)
- Errors (if any)

Note: **LLM** stands for Large Language Model.

Trace Signals to Record

Signal	Your Value
Total latency	___ ms
Input tokens	___
Output tokens	___
Tool calls	___
Memory operations	___
Errors	___

Lab Completion Checklist

Checklist (1/2)

- [] Team credentials loaded and identity verified
- [] Architecture components identified on the diagram
- [] Restricted IAM policy inspected on execution role
- [] Three TODOs completed in `agent.py`
- [] Agent deployed on Runtime: `READY` status
- [] Read-only tool invoked through Gateway: `GATEWAY_OK`
- [] Memory continuity verified across two sessions: `CONTINUITY_OK`
- [] End-to-end trace inspected in CloudWatch or fallback labeled clearly
- [] Remaining production controls explained in the share-out

Checklist (2/2)

DONE?

Help your neighbors or review the concept docs in the companion repo: [day1/concepts/](#)

Lab Complete

Next up: Kahoot Quiz 2, then Enterprise Practices